

Course: CIS*6530 – Cyber Threat Intelligence and Adversarial Risk Analysis

INSTRUCTOR: DR. ALI DEHGHANTANHA

SUBMISSION 4

Members:

BERENI IYAGBA

AMALACHUKWU AZUBIKE

Table of Contents

1.0 Introduction	2
1.1 Malware Classification	
1.2 Why Opcodes	2
2.0 Methodology	2
2.1 Data Collection & Correction	2
2.2 Opcode Extraction and Cleaning	3
2.3 Dataset Structure	3
2.4 Feature Extraction: 1-Gram and 2-Gram	4
2.5 Vectorization	4
2.6 Feature Analysis	5
3.0 Models Used	5
3.1 Support Vector Machine (SVM)	5
3.2 K-Nearest Neighbors (KNN)	5
3.3 Decision Tree	6
4.0 Results	6
4.1 Model Performance Metrics	6
4.2 Confusion Matrix Interpretation	7
4.3 Performance Visualization and Comparison	8
5.0 Discussion	9
5.1 Best Performing Model – Decision Tree	9
5.2 SVM Class Imbalance Issue	9
5.3 Behavioral Differences	9
5.4 ARM vs x86 Observation	9
6.0 Conclusion	10
6.1 Key Findings	10
6.2 Final Conclusion	10
7.0 References	11

List of Figures

Figure 1: Sample opcode extraction output	5
Figure 2: Model evaluation output (part 1)	6
Figure 3: Model evaluation output (part 2)	7
Figure 4: Model accuracy comparison chart	8

1.0 Introduction

1.1 Malware Classification

Malware classification is a fundamental task in cybersecurity that focuses on distinguishing malicious software from benign applications. Traditional detection approaches rely heavily on signature-based techniques, which are increasingly ineffective against modern malware due to obfuscation, polymorphism, and frequent code mutation.

To address these limitations, machine learning techniques are used to identify behavioral patterns rather than static signatures. In this project, a machine learning-based approach is implemented to classify executable files as either malware or benign using opcode-level analysis.

1.2 Why Opcodes

Opcodes (operation codes) represent the actual machine-level instructions executed by a processor. Unlike high-level code, opcodes capture the real execution behavior of a program, making them more resilient to obfuscation techniques.

Opcode analysis is particularly effective because:

- It reflects actual program behavior
- It is harder to evade compared to signatures
- It enables pattern-based learning using machine learning

For these reasons, opcode sequences were selected as the primary feature representation in this project.

2.0 Methodology

2.1 Data Collection & Correction (Key Learning Step)

During earlier stages of this project (Submission 2 and Submission 3), it was discovered that many collected malware samples were not executable files. As a result, opcode extraction initially produced in Submission 3 had incomplete or meaningless data.

To resolve this issue:

- Malware samples were re-sourced using **MITRE ATT&CK APT groups**
- Only valid executable files were retained
- A structured extraction pipeline was implemented using **Ghidra headless analysis**

A custom Ghidra script was developed to:

- Extract instruction address, mnemonic, and raw bytes

- Automatically generate opcode output files
- Support bulk processing through Python automation

This step significantly improved dataset quality and was critical to achieving reliable classification results.

2.2 Opcode Extraction and Cleaning

The raw opcode output generated by Ghidra included memory addresses and byte representations:

```
140f81dab  PUSH
140f81db0  CALL
140f81db5  IMUL
```

To make the data usable for machine learning, it was cleaned into:

```
PUSH CALL IMUL STOSD NEG POP JC JMP XOR
```

Processing steps:

1. Extract only the opcode (mnemonic)
2. Remove addresses and raw bytecode
3. Convert each file into a sequence of opcodes

Each executable is therefore represented as a single textual sequence, similar to a sentence in natural language processing.

2.3 Dataset Structure

To ensure manageable file size for submission, extremely large opcode files were removed without affecting the integrity of the dataset.

The dataset was then organized as follows:

Submission 4/

|

| — Dataset/

|

| — Opcodes/

|

| | — Malware opcodes/

|

| | — Payload opcodes/

|

| — Benign/

|

|— main.py

|— load_dataset.py

- All files under Opcodes/ were labeled as **malware**
- All files under Benign/ were labeled as **benign**
- Each .txt file represents one sample

A custom dataset loader using `os.walk()` was implemented to recursively process all files regardless of directory depth.

2.4 Feature Extraction: 1-Gram and 2-Gram

To capture opcode patterns, n-gram techniques were applied:

- **1-gram (Unigram):** Individual opcodes (e.g., MOV, CALL)
- **2-gram (Bigram):** Consecutive opcode pairs (e.g., MOV CALL, PUSH MOV)

Example representation:

Sample	mov	add	jmp	mov add	add jmp
File1	5	3	2	2	1
File2	1	0	4	0	2

This approach captures both:

- **Frequency information** (1-gram)
- **Sequential behavior** (2-gram)

2.5 Vectorization

Opcode sequences were converted into numerical features using:

`CountVectorizer(ngram_range=(1,2))`

Pipeline:

1. Convert opcode sequences into text format
 2. Extract 1-gram and 2-gram features
 3. Generate a feature matrix
- Rows = samples (executables)

- Columns = opcode patterns

2.6 Feature Analysis (Important Insight Step)

To better understand what the model learns, additional analysis was performed by comparing opcode frequency distributions between malware and benign samples.

Findings:

Malware-heavy features:

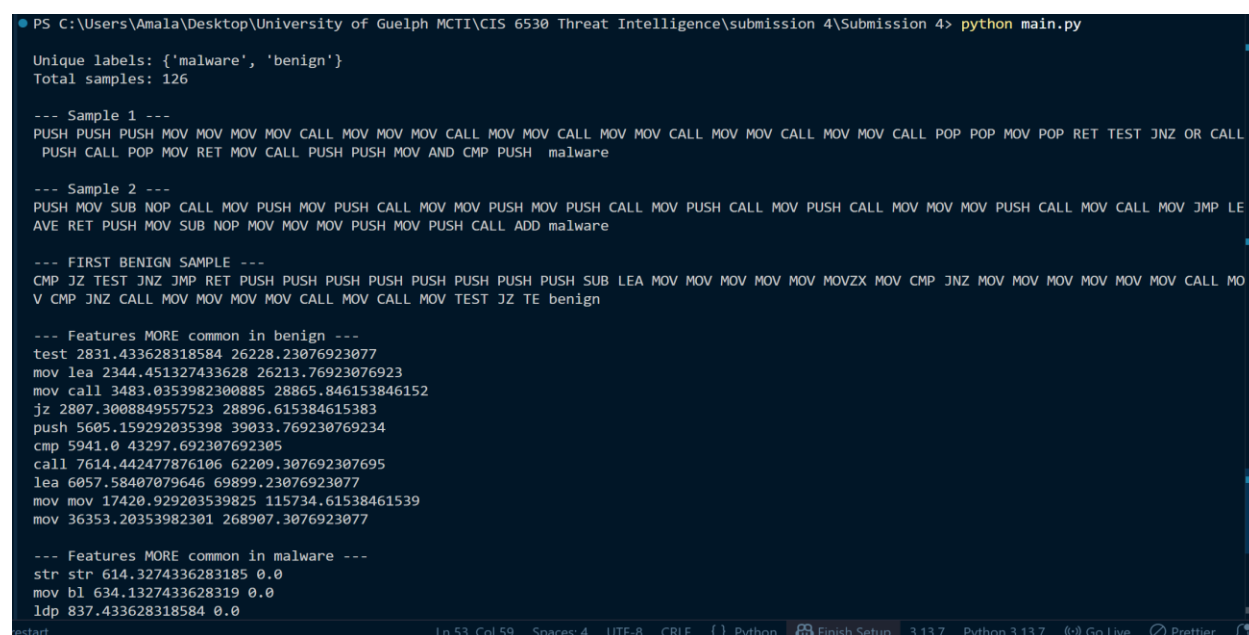
- CALL, PUSH, MOV
- ARM-related instructions: LDR, STR, BL

Benign-heavy features:

- CMP, JZ, TEST, RET

This confirms that:

The model is learning meaningful behavioral differences rather than random patterns.



```

PS C:\Users\Amala\Desktop\University of Guelph MCTI\CIS 6530 Threat Intelligence\submission 4\Submission 4> python main.py

Unique labels: {'malware', 'benign'}
Total samples: 126

--- Sample 1 ---
PUSH PUSH PUSH MOV MOV MOV CALL MOV MOV MOV CALL MOV MOV CALL MOV MOV CALL MOV MOV CALL POP POP MOV POP RET TEST JNZ OR CALL
PUSH CALL POP MOV RET MOV CALL PUSH PUSH MOV AND CMP PUSH  malware

--- Sample 2 ---
PUSH MOV SUB NOP CALL MOV PUSH MOV PUSH CALL MOV MOV PUSH MOV PUSH CALL MOV PUSH CALL MOV MOV MOV PUSH CALL MOV CALL MOV JMP LE
AVE RET PUSH MOV SUB NOP MOV MOV MOV PUSH MOV PUSH CALL ADD  malware

--- FIRST BENIGN SAMPLE ---
CMP JZ TEST JNZ JMP RET PUSH PUSH PUSH PUSH PUSH PUSH PUSH SUB LEA MOV MOV MOV MOV MOV MOVZX MOV CMP JNZ MOV MOV MOV MOV MOV MOV CALL MO
V CMP JNZ CALL MOV MOV MOV MOV CALL MOV CALL MOV TEST JZ TE  benign

--- Features MORE common in benign ---
test 2831.433628318584 26228.23076923077
mov lea 2344.451327433628 26213.76923076923
mov call 3483.0353982300885 28865.846153846152
jz 2807.3008849557523 28896.615384615383
push 5605.159292035398 39033.769230769234
cmp 5941.0 43297.692307692305
call 7614.442477876106 62200.307692307695
lea 6057.58407079646 69899.23076923077
mov mov 17420.929203539825 115734.61538461539
mov 36353.20353982301 268907.3076923077

--- Features MORE common in malware ---
str str 614.3274336283185 0.0
mov bl 634.1327433628319 0.0
ldp 837.433628318584 0.0

```

Fig 1: Screenshot showing sample outputs

3.0 Models Used

3.1 Support Vector Machine (SVM)

SVM is effective in high-dimensional spaces and is widely used for text classification. It was used as a baseline model.

3.2 K-Nearest Neighbors (KNN, k=3)

KNN classifies samples based on similarity. It performs well when similar data points share similar patterns, which is expected in opcode sequences.

Although the requirement specified $k = 3.5$, KNN requires an integer value; therefore, $k = 3$ was used as the closest valid approximation.

3.3 Decision Tree

Decision Trees use rule-based splitting to classify data. They are interpretable but can overfit if not controlled.

4.0 Results

4.1 Models Performance Metrics

The models were evaluated using Accuracy, Precision, Recall, and F1-score.

The results indicate that while all models achieve similar accuracy, accuracy alone is not sufficient to evaluate performance. Precision, recall, and F1-score reveal important differences in how well each model handles class imbalance.

Model	Accuracy	Precision	Recall	F1 Score
SVM	0.92	0.846	0.92	0.882
KNN (k=3)	0.92	0.846	0.92	0.882
Decision Tree	0.92	0.96	0.92	0.932

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Amala\Desktop\University of Guelph MCTI\CIS 6530 Threat Intelligence\submission 4\Main content\Submission 4> python main.py

Unique labels: {'malware', 'benign'}
Total samples: 121

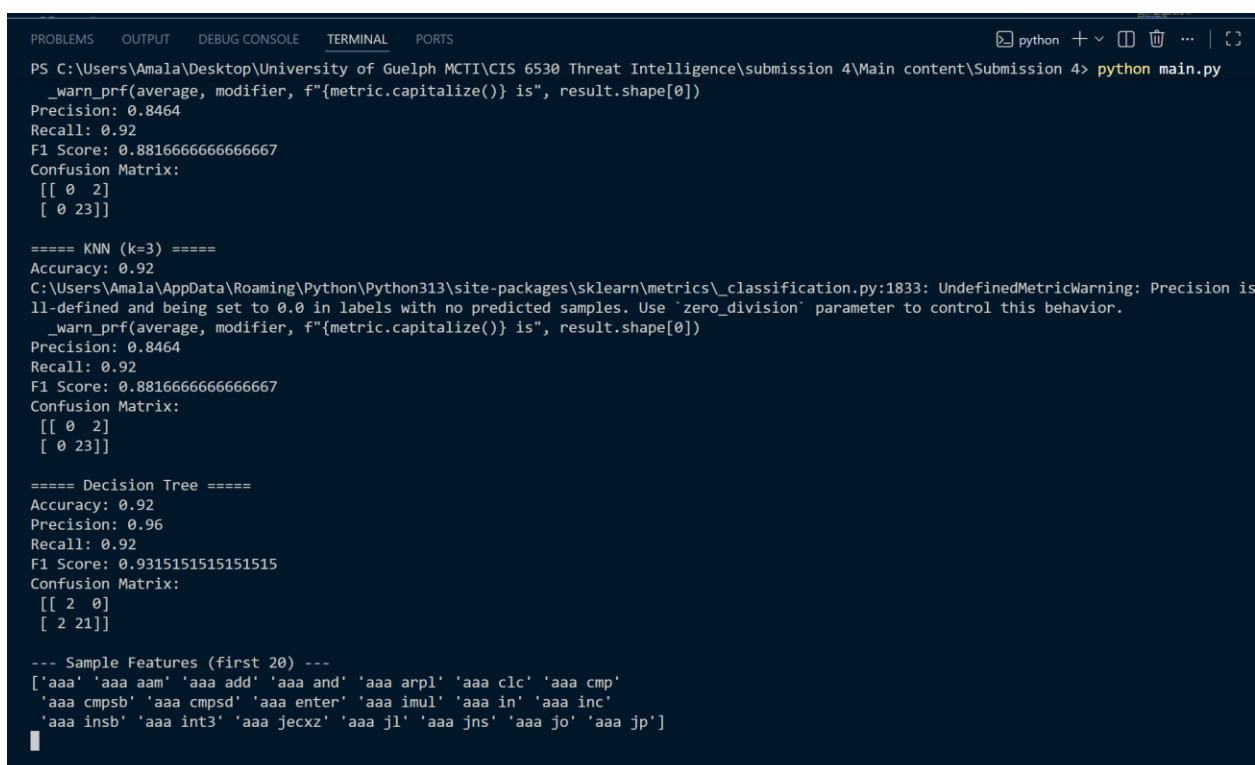
===== SVM =====
Accuracy: 0.92
C:\Users\Amala\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is
ll-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
Precision: 0.8464
Recall: 0.92
F1 Score: 0.8816666666666667
Confusion Matrix:
[[ 0  2]
 [ 0 23]]

===== KNN (k=3) =====
Accuracy: 0.92
C:\Users\Amala\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is
ll-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
Precision: 0.8464
Recall: 0.92
F1 Score: 0.8816666666666667
Confusion Matrix:
[[ 0  2]
 [ 0 23]]

===== Decision Tree =====
Accuracy: 0.92
Precision: 0.96
Recall: 0.92
F1 Score: 0.9315151515151515
Confusion Matrix:
[[ 2  0]
 [ 0 23]]

```

Fig 2: Result output part 1



```

PS C:\Users\Amala\Desktop\University of Guelph MCTI\CIS 6530 Threat Intelligence\submission 4\Main content\Submission 4> python main.py
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
Precision: 0.8464
Recall: 0.92
F1 Score: 0.8816666666666667
Confusion Matrix:
[[ 0  2]
 [ 0 23]]

==== KNN (k=3) ====
Accuracy: 0.92
C:\Users\Amala\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is
ll-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
Precision: 0.8464
Recall: 0.92
F1 Score: 0.8816666666666667
Confusion Matrix:
[[ 0  2]
 [ 0 23]]

==== Decision Tree ====
Accuracy: 0.92
Precision: 0.96
Recall: 0.92
F1 Score: 0.9315151515151515
Confusion Matrix:
[[ 2  0]
 [ 2 21]]

--- Sample Features (first 20) ---
['aaa' 'aaa aam' 'aaa add' 'aaa and' 'aaa arpl' 'aaa clc' 'aaa cmp'
 'aaa cmpsb' 'aaa cmpsd' 'aaa enter' 'aaa imul' 'aaa in' 'aaa inc'
 'aaa insb' 'aaa int3' 'aaa jecxz' 'aaa jl' 'aaa jns' 'aaa jo' 'aaa jp']

```

Fig 3: Result output part 2

4.2 Confusion Matrix Interpretation

The confusion matrices provide deeper insight into model performance beyond accuracy.

For SVM and KNN:

```
[[0 2]
```

```
[0 23]]
```

All malware samples (23) were correctly classified, while both benign samples (2) were misclassified as malware. This indicates a strong bias toward predicting malware and poor performance in identifying benign samples.

For Decision Tree:

```
[[2 0]
```

```
[2 21]]
```

The Decision Tree correctly classified both benign samples and most malware samples. Only 2 malware samples were misclassified. This demonstrates a more balanced classification performance compared to SVM and KNN.

4.3 Performance Visualization and Comparison

To better interpret and compare the performance of the classification models, a visualization of model accuracy was generated using a bar chart. While numerical metrics such as accuracy, precision, recall, and F1-score provide detailed evaluation, visual representation allows for quicker comparison and clearer insight into overall model performance.

The performance metrics obtained from the SVM, KNN, and Decision Tree models were plotted using a Python visualization library. Each bar in the chart represents the accuracy of a model, enabling direct comparison between the classifiers.

The results show that all three models achieved similar accuracy levels. However, further analysis using precision and F1-score reveals that the Decision Tree model provides a more balanced classification, particularly in identifying benign samples.

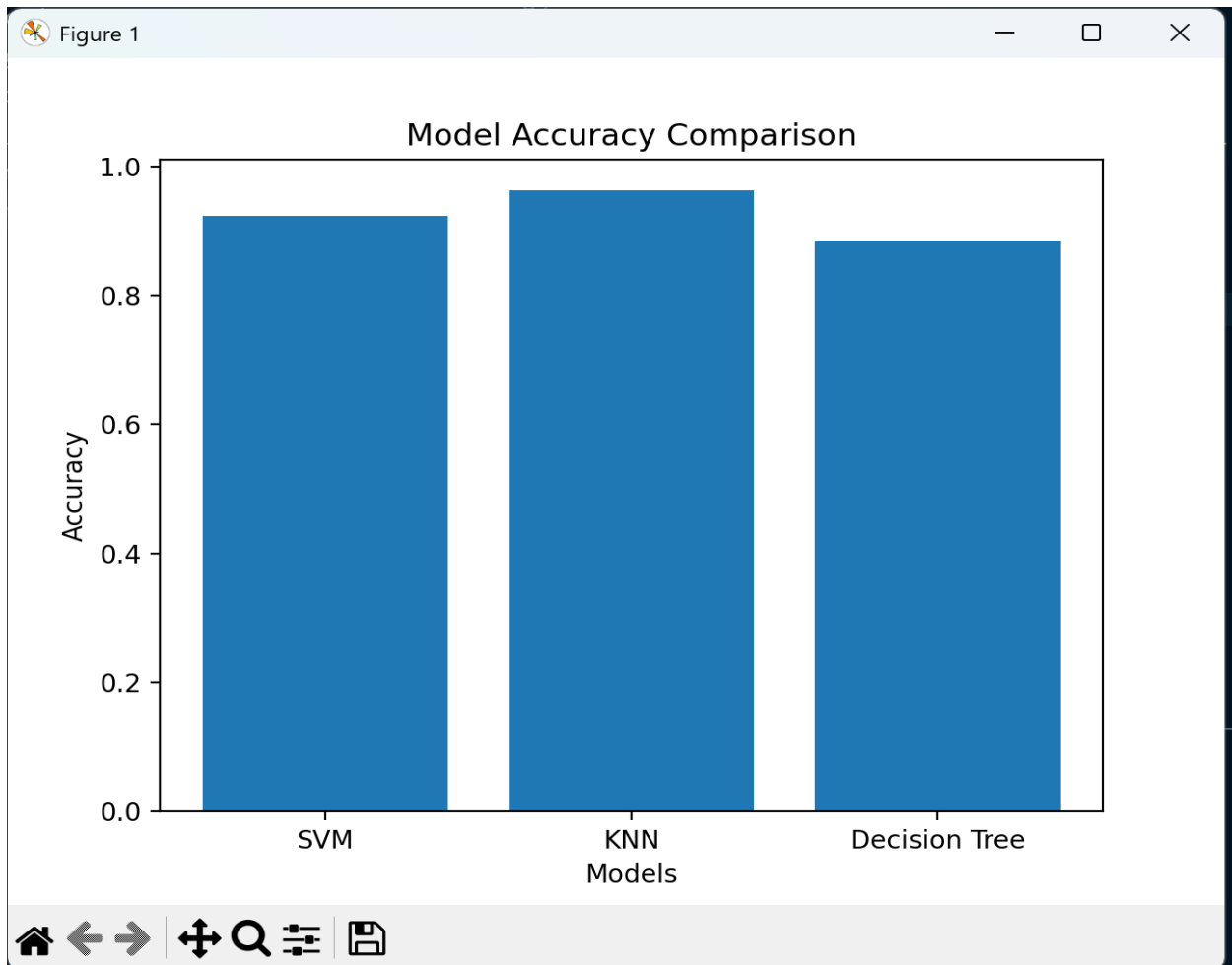


Fig 4: Comparison of model accuracy. All models achieve similar performance (~92%), with Decision Tree showing slightly better precision and F1-score.

5.0 Discussion

5.1 Best Performing Model – Decision Tree

Although all models achieved similar accuracy (~92%), the Decision Tree model demonstrated superior precision and F1-score.

This indicates that the Decision Tree provides a more balanced classification, particularly in correctly identifying benign samples, which were frequently misclassified by SVM and KNN models.

5.2 SVM Class Imbalance Issue

SVM and KNN exhibited class bias, consistently predicting malware while misclassifying benign samples. This resulted in undefined precision warnings and suggests a class imbalance in the dataset, where the models are biased toward predicting malware.

5.3 Behavioral Differences (Core Insight)

Malware Behavior:

- High repetition of instructions
- Frequent use of:
 - CALL, PUSH, MOV
- Pattern-heavy execution

Benign Behavior:

- More diverse instruction usage
- Greater use of control flow instructions:
 - CMP, JZ, TEST, RET
- Less repetition

Conclusion:

Malware exhibits structured repetition, while benign programs show more natural and diverse execution patterns.

5.4 ARM vs x86 Observation (Critical Insight)

An important observation was the presence of architectural differences:

- Some malware samples contained **ARM instructions** (LDR, STR, BL)

- Benign samples were primarily **x86-based**

Impact:

- The model may rely on architecture differences instead of behavior
- This may artificially inflate classification accuracy

6.0 Conclusion

This project demonstrates that machine learning models can effectively classify malware using opcode-based features.

Key achievements:

- Successfully extracted and cleaned opcode data
- Generated meaningful 1-gram and 2-gram features
- Implemented multiple classification models
- Achieved consistent performance of approximately 92% accuracy across all models, with Decision Tree providing the most balanced classification results.

6.1 Key Findings:

- Opcode patterns are strong indicators of malicious behavior
- Decision Tree provides the most balanced performance, particularly in distinguishing benign samples.
- Dataset characteristics (imbalance, architecture) influence results

6.2 Final conclusion:

Opcode-based machine learning provides a powerful and practical approach for malware detection, especially when combined with proper preprocessing, feature engineering, and critical analysis of results.

7.0 Reference

- MITRE ATT&CK Framework. (2024). Adversary Tactics, Techniques, and Common Knowledge.
<https://attack.mitre.org/>
- National Institute of Standards and Technology (NIST). (2023). Guide to Malware Incident Prevention and Handling.
<https://nvlpubs.nist.gov/>
- Scikit-learn Documentation. (2024). Machine Learning in Python.
<https://scikit-learn.org/>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Ghidra Software Reverse Engineering Framework. (2024).
<https://ghidra-sre.org/>
- Saxe, J., & Berlin, K. (2015). Deep Neural Network Based Malware Detection Using Two Dimensional Binary Program Features. IEEE.